

КРИПТОГРАФІЯ

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №1

Експериментальна оцінка ентропії на символ
джерела відкритого тексту

ФБ-43 Цвілій Влада

Мета роботи

Засвоєння понять ентропії на символ джерела та його надлишковості, вивчення та порівняння різних моделей джерела відкритого тексту для наближеного визначення ентропії, набуття практичних навичок щодо оцінки ентропії на символ джерела.

Хід роботи

Завантажила текст Змб:

 text.txt

29.06.2017 13:17

Текстовий докум...

3 047 КБ

Реалізація програми:

```
lab1.py > calc_h2
1  import collections
2  import math
3  import os
4  import random
5  import re
6
7
8  def find_file():
9      candidates = ["text.txt", "text.txt.txt", "text"]
10     for c in candidates:
11         if os.path.exists(c):
12             return c
13     for f in os.listdir("."):
14         if f.endswith(".txt"):
15             return f
16     raise FileNotFoundError("Не знайдено текстовий файл!")
17
18
19 def read_text(file_path):
20     for enc in ["utf-8", "windows-1251", "cp1251"]:
21         try:
22             with open(file_path, "r", encoding=enc) as f:
23                 content = f.read()
24                 if len(content) > 1000:
25                     return content
26         except Exception:
27             continue
28     raise ValueError("Не вдалося прочитати файл у відомих кодуваннях")
29
```

find_file() - шукає файл із текстом у папці. Вона перевіряє назви text.txt, text.txt.txt або будь-який інший файл, що закінчується на .txt. Це зроблено для того, щоб програма не видала помилку, якщо Windows випадково приховав розширення файлу.

`read_text(file_path)` - зчитує вміст файлу. Вона по черзі пробує різні кодування тексту, щоб правильно відкрити книгу

Правила попередньої обробки тексту, використані у програмі:

- Переведення тексту до нижнього регістру
- Допустимим алфавітом обрано 33 літери російської мови (а—я та ё). Усі символи, що не належать до цього алфавіту, замінено на символ пробілу.
- будь-яка послідовність із кількох пробілів або символів розділення підряд згорнута в один одинарний пробіл, а пробіли на початку та в кінці тексту видалено

тому:

- алфавіт тексту з пробілами становить 34 символи(33 літери+пробіл)
- тексту без пробілів відповідно 33 символи

```

30
31 def clean_text(raw_text):
32     text = raw_text.lower()
33     # Залишаємо тільки літери а-я та ё, все інше замінюємо на пробіл
34     text = re.sub(r"[^а-яё]", " ", text)
35     # Декілька пробілів замінюємо на один
36     text = re.sub(r"\s+", " ", text).strip()
37     return text, text.replace(" ", "")
38
39
40 def calc_h1(text):
41     total = len(text)
42     counts = collections.Counter(text)
43     entropy = 0
44     freqs = {}
45     for char, cnt in counts.items():
46         p = cnt / total
47         freqs[char] = p
48         entropy -= p * math.log2(p)
49     return entropy, freqs
50
51
52 def calc_h2(text, step=1):
53     bigrams = [text[i : i + 2] for i in range(0, len(text) - 1, step)]
54     total = len(bigrams)
55     counts = collections.Counter(bigrams)
56     entropy = 0
57     freqs = {}
58     for bg, cnt in counts.items():
59         p = cnt / total
60         freqs[bg] = p
61         entropy -= p * math.log2(p)
62     return entropy / 2.0, freqs
63
64

```

`clean_text(raw_text)` — очищає та нормалізує сирий текст за правилами з методички. Вона переводить усі літери в малі, викидає всі розділові знаки, цифри та латиницю (замінюючи їх на пробіл), а декілька пробілів підряд склеює в один. На виході функція видає одразу два варіанти тексту: перший — із пробілами, а другий — повністю без пробілів

`calc_h1(text)` — рахує ентропію окремих літер (H_1). Вона підраховує, скільки разів кожна літера зустрічається в тексті, ділить це число на загальну довжину тексту (отримує ймовірність p) і за формулою Шеннона вираховує значення ентропії. Також вона повертає список частот літер щоб потім знайти топ-5 найчастіших

`calc_h2(text, step=1)` — рахує ентропію біграм на символ (H_2). Вона ріже текст на пари літер. Параметр `step` задає спосіб нарізки:

- step=1 — біграми з перетином (зсув на 1 букву);
- step=2 — біграми без перетину (зсув на 2 букви).

Далі вона рахує ймовірності кожної пари, обчислює ентропію за формулою і в кінці ділить результат на 2 (тому що методичка вимагає знайти питому ентропію саме на один символ біграми)

```

65 def print_top5(freqs, name):
66     print(f"\nTop-5 ({name}):")
67     sorted_items = sorted(freqs.items(), key=lambda x: x[1], reverse=True)[:5]
68     for item, p in sorted_items:
69         display = f"'{item}'" if " " in item else item
70         print(f"    {display:<6} -> {p:.5f}")
71
72
73 def main():
74     filepath = find_file()
75     print(f"Обробка файлу: {filepath}")
76     raw = read_text(filepath)
77
78     txt_sp, txt_nosp = clean_text(raw)
79     print(f"Символів з пробілами: {len(txt_sp)}")
80     print(f"Символів без пробілів: {len(txt_nosp)}")
81

```

print_top5(freqs, name) — знаходить і друкує 5 найчастіших символів або біграм. Вона бере пораховані ймовірності, сортує їх від найбільшої до найменшої, відбирає перші 5 штук і виводить у термінал. Якщо символ — це пробіл, вона бере його в лапки, щоб його було добре видно в таблиці

```

82 # 1. ТЕКСТ З ПРОБІЛАМИ
83 print("\n" + "=" * 50)
84 print("1. ТЕКСТ З ПРОБІЛАМИ (34 символи: 33 літери + пробіл)")
85 print("=" * 50)
86 h1_sp, f1_sp = calc_h1(txt_sp)
87 h2_sp_cross, f2_sp_cross = calc_h2(txt_sp, step=1)
88 h2_sp_nocross, f2_sp_nocross = calc_h2(txt_sp, step=2)
89
90 h0_sp = math.log2(34)
91 r_h1_sp = 1 - (h1_sp / h0_sp)
92 r_h2_sp = 1 - (h2_sp_cross / h0_sp)
93
94 print(f"H0 (максимальна ентропія): {h0_sp:.4f} біт/символ")
95 print(f"H1: {h1_sp:.4f} біт/символ")
96 print(f"H2 (з перетином, крок 1): {h2_sp_cross:.4f} біт/символ")
97 print(f"H2 (без перетину, крок 2): {h2_sp_nocross:.4f} біт/символ")
98 print(f"Надлишковість за H1: {r_h1_sp:.4f} ({r_h1_sp*100:.2f}%)")
99 print(f"Надлишковість за H2: {r_h2_sp:.4f} ({r_h2_sp*100:.2f}%)")
100
101 print_top5(f1_sp, "символи (з пробілами)")
102 print_top5(f2_sp_cross, "біграми з перетином (з пробілами)")
103 print_top5(f2_sp_nocross, "біграми без перетину (з пробілами)")
104

```

Рядки 86–88: викликає функції для розрахунку ентропії літер, біграм із перетином та біграм без перетину

Рядок 90 ($h0_sp = \text{math.log2}(34)$): вираховує максимально можливу ентропію H_0 для алфавіту з 34 символів (33 російські літери + пробіл), якби всі вони зустрілися з однаковою ймовірністю.

Рядки 91–92 (r_h1_sp , r_h2_sp): рахує надлишковість мови R за формулою з методички: $R = 1 - \frac{H}{H_0}$

Рядки 94–103: друкує всі отримані цифри та топ-5 найчастіших літер і пар у термінал

```

104
105 # 2. ТЕКСТ БЕЗ ПРОБІЛІВ
106 print("\n" + "=" * 50)
107 print("2. ТЕКСТ БЕЗ ПРОБІЛІВ (33 символи: літери а-я, ё)")
108 print("=" * 50)
109 h1_nosp, f1_nosp = calc_h1(txt_nosp)
110 h2_nosp_cross, f2_nosp_cross = calc_h2(txt_nosp, step=1)
111 h2_nosp_nocross, f2_nosp_nocross = calc_h2(txt_nosp, step=2)
112
113 h0_nosp = math.log2(33)
114 r_h1_nosp = 1 - (h1_nosp / h0_nosp)
115 r_h2_nosp = 1 - (h2_nosp_cross / h0_nosp)
116
117 print(f"H0 (максимальна ентропія): {h0_nosp:.4f} біт/символ")
118 print(f"H1: {h1_nosp:.4f} біт/символ")
119 print(f"H2 (з перетином, крок 1): {h2_nosp_cross:.4f} біт/символ")
120 print(f"H2 (без перетину, крок 2): {h2_nosp_nocross:.4f} біт/символ")
121 print(f"Надлишковість за H1: {r_h1_nosp:.4f} ({r_h1_nosp*100:.2f}%)")
122 print(f"Надлишковість за H2: {r_h2_nosp:.4f} ({r_h2_nosp*100:.2f}%)")
123
124 print_top5(f1_nosp, "символи (без пробілів)")
125 print_top5(f2_nosp_cross, "біграми з перетином (без пробілів)")
126 print_top5(f2_nosp_nocross, "біграми без перетину (без пробілів)")
127

```

Рядок 113 (`math.log2(33)`): береться логарифм від 33, а не від 34, тому що пробілу в алфавіті більше немає Теоретичний максимум ентропії H_0 стає трохи меншим

Рядки 114–115: знову рахує надлишковість тексту R , але вже для моделі без пробілів.

Рядки 124–126: виводить топ-5 найчастіших літер і біграм. У топі замість пробілу на перше місце виходить буква «о», а найчастішою біграмою стає пара «то»

```

128 # 3. ДОСЛІДЖЕННЯ РОЗПОДІЛІВ (РОЗДІЛ 3)
129 print("\n" + "=" * 50)
130 print("РОЗДІЛ 3. ВПЛИВ РОЗПОДІЛУ СИМВОЛІВ")
131 print("=" * 50)
132 n = 100000
133 alpha = list(set(txt_nosp))
134 seq_a = txt_nosp[:n]
135 seq_b = "a" * n
136 seq_v = "".join(random.choices(alpha, k=n))
137
138 h1_a, _ = calc_h1(seq_a)
139 h1_b, _ = calc_h1(seq_b)
140 h1_v, _ = calc_h1(seq_v)
141
142 print(f"Послідовність А (природний текст):          H1 = {h1_a:.4f}")
143 print(f"Послідовність Б (один символ 'a'):          H1 = {h1_b:.4f}")
144 print(f"Послідовність В (рівноймовірна випадкова): H1 = {h1_v:.4f}")
145

```

створюємо три штучні тексти однакової довжини по 100 000 символів ($n = 100000$):

`seq_a` — відрізає перші 100 000 символів реальної книги. Тут розподіл літер нерівномірний (природний), ентропія виходить близько 4.46

`seq_b` — генерує рядок лише з однієї літери «а». Невизначеності взагалі немає, ми на 100% знаємо наступний символ, тому ентропія виходить рівно 0.0000

`seq_v` — генерує абсолютно випадкові літери через `random.choices`, де кожна літера має однаковий шанс випасти. Це стан максимального хаосу, тому ентропія виходить максимально можливою: 5.0441

```

145
146 # 4. ДОСЛІДЖЕННЯ ЗАЛЕЖНОСТЕЙ (РОЗДІЛ 4)
147 print("\n" + "=" * 50)
148 print("РОЗДІЛ 4. ЗАЛЕЖНОСТІ МІЖ СИМВОЛАМИ")
149 print("=" * 50)
150 half = n // 2
151 lst_g = ["a"] * half + ["b"] * half
152 random.shuffle(lst_g)
153 seq_g = "".join(lst_g)
154 seq_d = "ab" * half
155
156 h1_g, _ = calc_h1(seq_g)
157 h2_g_c, _ = calc_h2(seq_g, step=1)
158 h2_g_nc, _ = calc_h2(seq_g, step=2)
159
160 h1_d, _ = calc_h1(seq_d)
161 h2_d_c, _ = calc_h2(seq_d, step=1)
162 h2_d_nc, _ = calc_h2(seq_d, step=2)
163
164 print(
165     f"Послідовність Г (випадковий порядок 'a' та 'b'): \n H1 = {h1_g:.4f}, H2(перетин) = {h2_g_c:.4f}, H2(без перетину) = {h2_g_nc:.4f}"
166 )
167 print(
168     f"Послідовність Д (періодична 'абаб...'): \n H1 = {h1_d:.4f}, H2(перетин) = {h2_d_c:.4f}, H2(без перетину) = {h2_d_nc:.4f}"
169 )
170
171
172 if __name__ == "__main__":
173     main()

```

PS C:\Users\Lenovo\Desktop\crypto_lab1> & C:\Users\Lenovo\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11.0.0_x-ww\python.exe

Обробка файлу: text.txt

Символів з пробілами: 2822066

Символів без пробілів: 2362361

=====

1. ТЕКСТ З ПРОБІЛАМИ (34 символи: 33 літери + пробіл)

=====

H0 (максимальна ентропія): 5.0875 біт/символ

H1: 4.3777 біт/символ

H2 (з перетином, крок 1): 3.9662 біт/символ

H2 (без перетину, крок 2): 3.9660 біт/символ

Надлишковість за H1: 0.1395 (13.95%)

Надлишковість за H2: 0.2204 (22.04%)

Топ-5 (символи (з пробілами)):

' ' -> 0.16290

o -> 0.09630

e -> 0.06956

a -> 0.06848

и -> 0.05668

Топ-5 (біграми з перетином (з пробілами)):

'o ' -> 0.02209

'и ' -> 0.01894

'e ' -> 0.01723

'a ' -> 0.01660

' c' -> 0.01571

Топ-5 (біграми без перетину (з пробілами)):

'o ' -> 0.02218

'и ' -> 0.01902

'e ' -> 0.01727

'a ' -> 0.01677

' c' -> 0.01584

2. ТЕКСТ БЕЗ ПРОБІЛІВ (33 символи: літери а-я, ё)

Н₀ (максимальна ентропія): 5.0444 біт/символ
Н₁: 4.4636 біт/символ
Н₂ (з перетином, крок 1): 4.1415 біт/символ
Н₂ (без перетину, крок 2): 4.1412 біт/символ
Надлишковість за Н₁: 0.1151 (11.51%)
Надлишковість за Н₂: 0.1790 (17.90%)

Топ-5 (символи (без пробілів)):

о -> 0.11504
е -> 0.08310
а -> 0.08181
и -> 0.06771
н -> 0.06470

Топ-5 (біграми з перетином (без пробілів)):

то -> 0.01756
ст -> 0.01351
на -> 0.01247
ов -> 0.01160
го -> 0.01058

Топ-5 (біграми без перетину (без пробілів)):

то -> 0.01751
ст -> 0.01350
на -> 0.01245
ов -> 0.01154
го -> 0.01060

Порівняння результатів для текстів з пробілами і без:

Варіант тексту	H_2 з перетином (крок 1)	H_2 без перетину (крок 2)	Абсолютна різниця
З пробілами	3.9662 біт/символ	3.9660 біт/символ	0.0002
Без пробілів	4.1415 біт/символ	4.1412 біт/символ	0.0003

Отримані значення практично збігаються - розбіжність становить лише 2–3 десятитисячних біта

Який спосіб використовує більше статистичних даних?

Спосіб із перетином (зсув на 1 символ) використовує приблизно вдвічі більше даних.

Для тексту довжиною N символів спосіб із перетином формує $N - 1$ біграм, у нашому випадку — близько 2.82 млн пар

Спосіб без перетину (зсув на 2 символи) формує лише $N/2$ біграм, близько 1.41 млн пар, тобто відкидає рівно половину можливих сусідніх сполучень

```
=====
РОЗДІЛ 3. ВПЛИВ РОЗПОДІЛУ СИМВОЛІВ
=====
```

```
Послідовність А (природний текст):      Н1 = 4.4674
Послідовність Б (один символ 'а'):      Н1 = 0.0000
Послідовність В (рівноймовірна випадкова): Н1 = 5.0441
=====
```

Очікуваний порядок зростання H_1 : Б < А < В

послідовність Б: $H = 0$, адже текст складається з одного повторюваного символу.
Ми чітко знаємо який символ буде наступним

Послідовність А: H буде середня, адже різні літери зустрічаються з різною ймовірністю. Така нерівномірність зменшує невизначеність, тому ентропія природного тексту завжди суттєво нижча за теоретичний максимум

Послідовність В: Це штучно згенерована випадкова послідовність, де всі літери з'являються з однаковою ймовірністю, тут ентропія досягає свого абсолютного теоретичного максимуму

Результати розрахунку:

Послідовність А: $H_1 = 4.4674$ біт/символ

Послідовність Б: $H_1 = 0.0000$ біт/символ

Послідовність В: $H_1 = 5.0441$ біт/символ

Попередній прогноз повністю підтвердився: $H_1(Б) < H_1(А) < H_1(В)$

Вплив розподілу: нерівномірність розподілу ймовірностей у мові суттєво зменшує ентропію. Чим ближчий розподіл до випадкового, тим вища ентропія; а вироджений розподіл дає нульову ентропію

```

=====
РОЗДІЛ 4. ЗАЛЕЖНОСТІ МІЖ СИМВОЛАМИ
=====
Послідовність Г (випадковий порядок 'а' та 'б'):
  Н1 = 1.0000, Н2(перетин) = 1.0000, Н2(без перетину) = 1.0000
Послідовність Д (періодична 'абаб...'):
  Н1 = 1.0000, Н2(перетин) = 0.5000, Н2(без перетину) = 0.0000
PS C:\Users\Lenovo\Desktop\crypto_lab1>

```

Прогноз для Г і Д:

1. чи повинні суттєво відрізнятися значення H_1 для послідовностей Г і Д;

Ні, не повинні. Частоти окремих літер «а» та «б» в обох послідовностях однакові, а ентропія H_1 враховує лише ймовірності символів і взагалі не залежить від їхнього порядку

2. чи повинні суттєво відрізнятися значення H_2 ;

Так, повинні суттєво відрізнятися. Модель біграм враховує зв'язки між сусідніми літерами, а порядок символів у випадковій послідовності Г та циклічній Д різний

3. для якої послідовності ви очікуєте менше значення H_2 .

Для періодичної послідовності Д. У ній наступний символ повністю передбачуваний за попереднім, тому невизначеність прямує до нуля

4. чи відрізнятимуться H_2 обчислене для біграм що перетинаються і що не перетинаються

Для тексту Г: практично не відрізнятимуться

Для тексту Д: відрізнятимуться суттєво. Без перетину текст розіб'ється на абсолютно однакові пари «аб», і невизначеність зникне. З перетином утворюються два типи пар («аб» і «ба»), тому H_2 дорівнюватиме 0.5

Порівняння результатів з прогнозом:

Отримані результати повністю підтвердили прогноз:

Для обох послідовностей $H_1 = 1.0000$ біт/символ

Для послідовності Г (випадкової): H_2 (з перетином) = 1.0000, H_2 (без перетину) = 1.0000

Для послідовності Д (періодичної): H_2 (з перетином) = 0.5000, H_2 (без перетину) = 0.0000

- Чому тексти мають однакові значення H_1 :
Ентропія H_1 залежить виключно від частоти появи окремих літер і взагалі не враховує їхній порядок. Оскільки і в Г, і в Д літери «а» та «б» взяті порівну, для моделі H_1 ці тексти абсолютно ідентичні
- Чому їхні значення H_2 суттєво відрізняються:
Ентропія H_2 враховує зв'язок між парою сусідніх літер. У випадковій послідовності Г пари літер хаотичні, тому невизначеність залишається максимальною. У періодичній послідовності Д наступний символ на 100% визначений попереднім. При розбитті без перетину весь текст складається лише з однієї пари «аб», тому невизначеність зникає і $H_2 = 0.0000$
- Яку інформацію дозволяє врахувати модель біграм:
Модель біграм дозволяє врахувати статистичні залежності між сусідніми символами. Вона показує порядок слідування літер та правила їх поєднання, виявляючи структуру тексту, яку модель окремих символів побачити не може

5. Експериментальна оцінка ентропії джерела

Для 10 символів:

Лабораторная работа №1

Произвольная часть текста:
_силы_тяг

Использованные буквы:

Порядок n-граммы:
5 символов
10 символов
15 символов
20 символов
25 символов
30 символов
35 символов
40 символов
45 символов
50 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 57

Поле ввода символов:

Продолжить Другой

Неравенство для энтропии:
4,20448818427488 < H < 4,16599515489879

Двоичная таблица угаданных символов:
00000000000000000000000000000001
00000000000100000000000000000000
00000000000001000000000000000000
00000000000000000000000000000000
00000001000000000000000000000000
~~~~~

Вероятности:  
q[1] = 0,0535714  
q[2] = 0,0714285  
q[3] = 0,0714285  
q[4] = 0  
q[5] = 0,0357142  
q[6] = 0,1428571  
q[7] = 0,125  
q[8] = 0,0714285  
q[9] = 0,0357142  
q[10] = 0,035714  
q[11] = 0,017857  
q[12] = 0,035714  
q[13] = 0  
q[14] = 0,017857  
q[15] = 0,053571  
q[16] = 0,017857  
q[17] = 0,017857  
q[18] = 0  
q[19] = 0,017857  
q[20] = 0,035714  
q[21] = 0  
q[22] = 0,017857  
q[23] = 0,017857  
q[24] = 0  
q[25] = 0,017857  
q[26] = 0  
q[27] = 0  
q[28] = 0,017857  
q[29] = 0,053571  
q[30] = 0  
q[31] = 0  
q[32] = 0,017857

Строка состояния:

Для 20 символів:

[illegible]

Для 30 символів:

**Лабораторная работа №1**

---

Произвольная часть текста:  
анном\_конкретном\_случае\_у\_нег

Использованные буквы:

---

Порядок n-граммы:

- 5 символов
- 10 символов
- 15 символов
- 20 символов
- 25 символов
- 30 символов
- 35 символов
- 40 символов
- 45 символов
- 50 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 52

Поле ввода символов:

Продолжить Другой

Неравенство для энтропии:  
 $2,82434780206318 < H < 3,32832975331199$

Двоичная таблица угаданных символов:

|                                  |
|----------------------------------|
| 01000000000000000000000000000000 |
| 01000000000000000000000000000000 |
| 10000000000000000000000000000000 |
| 10000000000000000000000000000000 |
| 10000000000000000000000000000000 |
| ~~~~~                            |

Вероятности:

|                  |
|------------------|
| q[1] = 0,3137254 |
| q[2] = 0,1764705 |
| q[3] = 0,0196078 |
| q[4] = 0,0392156 |
| q[5] = 0,0588235 |
| q[6] = 0,0196078 |
| q[7] = 0,0196078 |
| q[8] = 0,0392156 |
| q[9] = 0         |
| q[10] = 0        |
| q[11] = 0        |
| q[12] = 0        |
| q[13] = 0,039215 |
| q[14] = 0,058823 |
| q[15] = 0        |
| q[16] = 0,058823 |
| q[17] = 0,019607 |
| q[18] = 0,019607 |
| q[19] = 0,019607 |
| q[20] = 0        |
| q[21] = 0        |
| q[22] = 0,039215 |
| q[23] = 0        |
| q[24] = 0        |
| q[25] = 0        |
| q[26] = 0        |
| q[27] = 0        |
| q[28] = 0        |
| q[29] = 0,058823 |
| q[30] = 0        |
| q[31] = 0        |
| q[32] = 0        |

Строка состояния:

Результати:

| Оцінка | Отримане значення     |
|--------|-----------------------|
| H1     | 4.3777                |
| H2     | 3.9662                |
| H(10)  | $4.1660 < H < 4.2045$ |
| H(20)  | $3.6668 < H < 3.8233$ |
| H(30)  | $2.8243 < H < 3.3283$ |

1. Чому наведені оцінки ентропії не повинні обов'язково збігатися:

Вони базуються на різних наближених моделях джерела тексту, які враховують різну кількість інформації про контекст. Н1 моделює текст як випадковий набір ізольованих літер без пам'яті, Н2 пам'ятає лише один попередній символ, а експерименти Н(10), Н(20) та Н(30) враховують передісторію довжиною 10, 20 і 30 символів відповідно. Оскільки літери в словах і реченнях тісно пов'язані між собою, моделі з різною глибиною пам'яті дають різну оцінку невизначеності

2. Які статистичні залежності враховуються при визначенні  $H_1$  та  $H_2$ :

При визначенні  $H_1$  враховується лише одновимірний розподіл частот — тобто скільки разів кожна окрема літера взагалі зустрічається в тексті, без огляду на сусідні літери

При визначенні  $H_2$  додатково враховуються парні залежності

3. Чому врахування довших залежностей між символами може змінювати оцінку ентропії природної мови:

У природній мові правила діють не лише між сусідніми літерами, але й на рівні цілих слів, закінчень і речень. Довгий контекст у 10–30 символів дозволяє людині або алгоритму розпізнати корінь слова чи навіть здогадатися про наступне слово за змістом фрази. Це різко звужує вибір наступної літери до 1–2 варіантів, що кардинально зменшує ентропію

4. Яку тенденцію можна спостерігати при переході від  $H_1$  до оцінок, що враховують дедалі довший контекст:

Спостерігається чітка тенденція до постійного спадання оцінки ентропії:  $H_1 > H_2 > H(10) > H(20) > H(30)$ . У нашому експерименті значення поступово впали від 4.38 біт/символ до приблизно 3.08 біт/символ. Це доводить: чим більше передісторії нам відомо, тим вищою стає передбачуваність тексту

## 6. Оцінювання надлишковості джерела

Для кожної дослідженої моделі обчислимо коефіцієнт надлишковості за формулою:

$R = 1 - (H / H_{\max})$ , де теоретичний максимум для нашого алфавіту (34 символи) становить  $H_{\max} = \log_2(34) = 5.0875$  біт/символ

| Модель джерела                   | Ентропія $H$<br>(біт/символ) | Розрахунок $R$          | Надлишковість (%) |
|----------------------------------|------------------------------|-------------------------|-------------------|
| Окремі символи ( $H_1$ )         | 4.3777                       | $1 - (4.3777 / 5.0875)$ | 13.95%            |
| Біграми з перетином ( $H_2$ )    | 3.9662                       | $1 - (3.9662 / 5.0875)$ | 22.04%            |
| Контекст 10 символів ( $H(10)$ ) | 4.1850                       | $1 - (4.1850 / 5.0875)$ | 17.74%            |
| Контекст 20 символів ( $H(20)$ ) | 3.7450                       | $1 - (3.7450 / 5.0875)$ | 26.39%            |
| Контекст 30 символів ( $H(30)$ ) | 3.0760                       | $1 - (3.0760 / 5.0875)$ | 39.54%            |

Отримані розрахунки показують, що при переході від моделі окремих символів до моделей із довгим контекстом оцінка надлишковості російського тексту зростає майже втричі (з 14% до 40%). Це пояснюється наявністю граматичних та лексичних правил у мові, які роблять велику частину тексту заздалегідь передбачуваною

Порівняння отриманих оцінок:

- **Як змінюється оцінка надлишковості при врахуванні залежностей між символами:**

При врахуванні залежностей між символами оцінка надлишковості постійно зростає. Якщо для окремих літер (модель H1) надлишковість становить лише близько 14%, то для пар літер (H2) вона піднімається до 22%, а при врахуванні довгого контексту слів і речень (H30) сягає майже 40%. Це свідчить про те, що чим глибші зв'язки між символами ми враховуємо, тим менше випадковості залишається в тексті і тим більша його частина є передбачуваною

- **Чому природна мова має ненульову надлишковість:**

Природна мова має ненульову надлишковість через суворі правила побудови слів і речень: орфографію, граматику та обмежений словниковий запас. Крім того, надлишковість виконує життєво важливу функцію захисту від завад і помилок: саме завдяки їй людина здатна легко читати текст з одруківками, розуміти розмиті слова або чути мову в шумному середовищі

- **Який зв'язок існує між передбачуваністю наступних символів, ентропією та надлишковістю джерела:**

Між цими поняттями існує прямий зв'язок:

- Що вища передбачуваність наступного символу, тим менше в ньому несподіванки.
- Менша несподіванка означає нижчу ентропію тексту
- А чим нижча ентропія відносно теоретичного максимуму, тим вищою є надлишковість джерела

## Аналіз результатів

### 1. Чому природний текст має меншу ентропію на символ, ніж випадкова рівноймовірна послідовність символів того самого алфавіту?

Тому що в природній мові літери зустрічаються вкрай нерівномірно (наприклад, літера «о» має вищу частоту ніж літери «ф» чи «ъ»). Будь-яка нерівномірність зменшує невизначеність. Натомість у випадковій послідовності всі символи рівноймовірні, що дає максимальний хаос.

Підтвердження з розділу 3: ентропія природного тексту склала  $H_1 = 4.4674$  біт/символ, що суттєво менше за випадкову рівноймовірну послідовність В, де  $H_1 = 5.0441$  біт/символ

### 2. Яку інформацію про джерело враховує $H_2$ , але не враховує $H_1$ ?

Модель  $H_1$  враховує лише загальну частоту появи окремої літери в тексті.

Модель  $H_2$  додатково враховує статистичні зв'язки між сусідніми літерами: правила сполучуваності голосних і приголосних, типові префікси та закінчення, а також неможливі комбінації букв

Підтвердження з розділу 2: саме завдяки врахуванню найчастіших пар літер ентропія природного тексту впала з  $H_1 = 4.38$  до  $H_2 = 3.97$  біт/символ. А в Розділі 4 модель  $H_2$  наочно виявила жорсткий зв'язок між літерами в тексті «абаб...» ( $H_2$  впала до 0), чого модель  $H_1$  взагалі не помітила

### 3. Чи можуть два тексти мати однакове $H_1$ , але різне $H_2$ ? Наведіть приклад із виконаних експериментів.

Так, можуть. Це наочно продемонстрував наш експеримент із Розділу 4:

Тексти Г (випадковий порядок букв «а» і «б») та Д (періодична структура «абабаб...») мають однаковий склад символів (по 50%), тому їхня ентропія окремих літер повністю однакова:  $H_1 = 1.0000$  біт/символ. Проте через наявність жорсткої закономірності в тексті Д його ентропія біграм без перетину впала до  $H_2 = 0.0000$ , тоді як для випадкового тексту Г вона залишилася максимальною:  $H_2 = 1.0000$

### 4. Як вилучення пробілів вплинуло на отримані значення ентропії? Поясніть отриманий результат.

У нашому розрахунку (Розділ 2) вилучення пробілів призвело до зростання ентропії  $H_1$  з 4.3777 до 4.4636 біт/символ (а  $H_2$  зросло з 3.9662 до 4.1415) тому що пробіл був найбільш частим символом тексту. Коли ми прибрали цей домінуючий символ, розподіл імовірностей літер, що залишилися, став більш рівномірним, через що середня невизначеність на символ зросла

## **5. Чому оцінки $H_2$ , отримані за біграмами, що перетинаються та не перетинаються, можуть дещо відрізнятися?**

Через різницю в обсязі статистичних даних (Розділ 2). Спосіб із перетином аналізує кожну пару сусідів і дає вибірку в 2.82 млн біграм, а спосіб без перетину бере лише пари на парних позиціях (вибірка вдвічі менша). Втрата половини позицій може викликати локальні коливання частот. Проте на нашому великому тексті оцінки майже ідеально зійшлися: 3.9662 проти 3.9660

## **6. Чому значення $H_1$ , $H_2$ , $H(10)$ , $H(20)$ та $H(30)$ відрізняються?**

Вони відрізняються через різну глибину врахованого контексту (Розділ 5).  $H_1$  розглядає літери ізольовано,  $H_2$  пам'ятає 1 попередній символ, а в експерименті Шеннона контекст сягає 10, 20 і 30 символів (цілі слова та фрази). Що більший контекст ми знаємо, то менше варіантів залишається для наступної літери, тому ентропія закономірно спадає: 4.38 -> 3.97 -> 4.19 -> 3.75 -> 3.08 біт/символ.

## **7. Який із проведених експериментів найкраще демонструє наявність залежностей між послідовними символами?**

Найкраще це доводять два експерименти:

1. Експеримент із Розділу 4, де регулярність чергування обвалила ентропію  $H_2$  з 1.0 до 0.0 при незмінному  $H_1$
2. Експеримент із Розділу 5: при збільшенні підказки з 10 до 30 символів частота правильного вгадування літери з першої спроби зросла з 5% до понад 31%, а верхня межа ентропії знизилася з 4.20 до 3.33 біт

## **8. Як отримані результати пов'язані з можливістю стиснення природного тексту без втрати інформації?**

Зв'язок прямий: ентропія показує, скільки реально бітів потрібно на один символ, щоб зберегти весь зміст тексту. У Розділі 6 ми визначили, що якби всі літери були випадковими, на кожну треба було б витратити 5.09 біт. Але наш експеримент із вгадуванням слів (Розділ 5) показав, що завдяки правилам мови реальна ентропія падає до 3.08 біт на символ ( $H_{30}$ ), тобто майже 40% тексту – надлишковість

## **Висновок**

У ході лабораторної роботи було експериментально оцінено ентропію та надлишковість російського письмового тексту за різними моделями джерела. Результати продемонстрували закономірне зниження оцінки ентропії зі зростанням довжини врахованого контексту — від 4.38 біт/символ для моделі

ізолюваних літер (H1) до 3.08 біт/символ при контексті у 30 символів (H30). Експерименти зі штучними вибірками довели, що модель біграм (H2), на відміну від моделі H1, здатна виявляти внутрішні структурні залежності між символами навіть при однакових частотах літер. Встановлено, що нерівномірний розподіл символів та наявність граматичних зв'язків зумовлюють високу надлишковість природного тексту (близько 40%), що теоретично підтверджує можливість його ефективного стиснення алгоритмами архівації майже вдвічі без будь-якої втрати змісту